

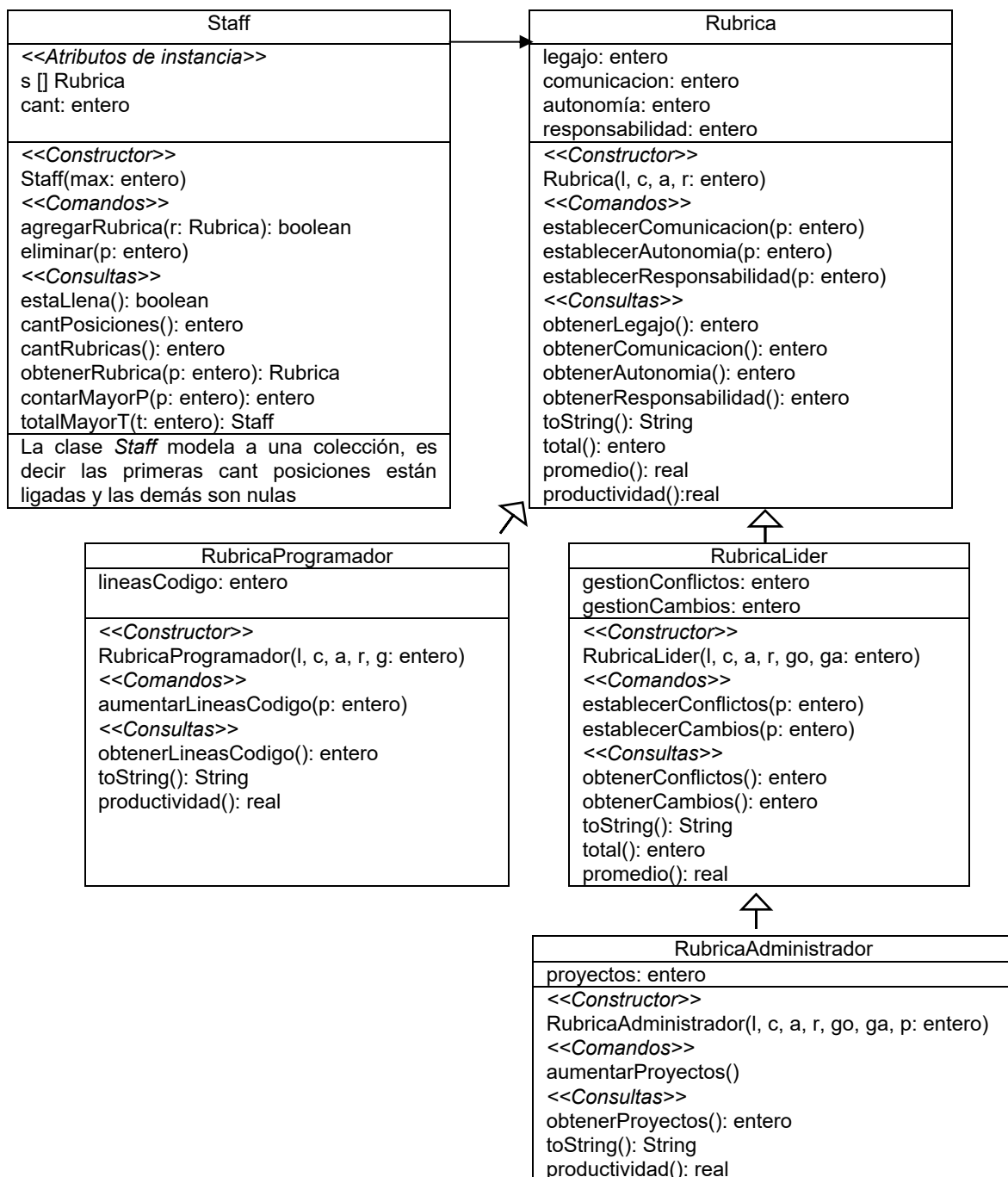


Actividades prácticas Estructuras de datos polimórficas

Objetivos

- Implementar un diagrama de clases relacionadas por asociación, dependencia y herencia simple.
- Comprender el impacto de la herencia y el polimorfismo sobre la productividad de software.

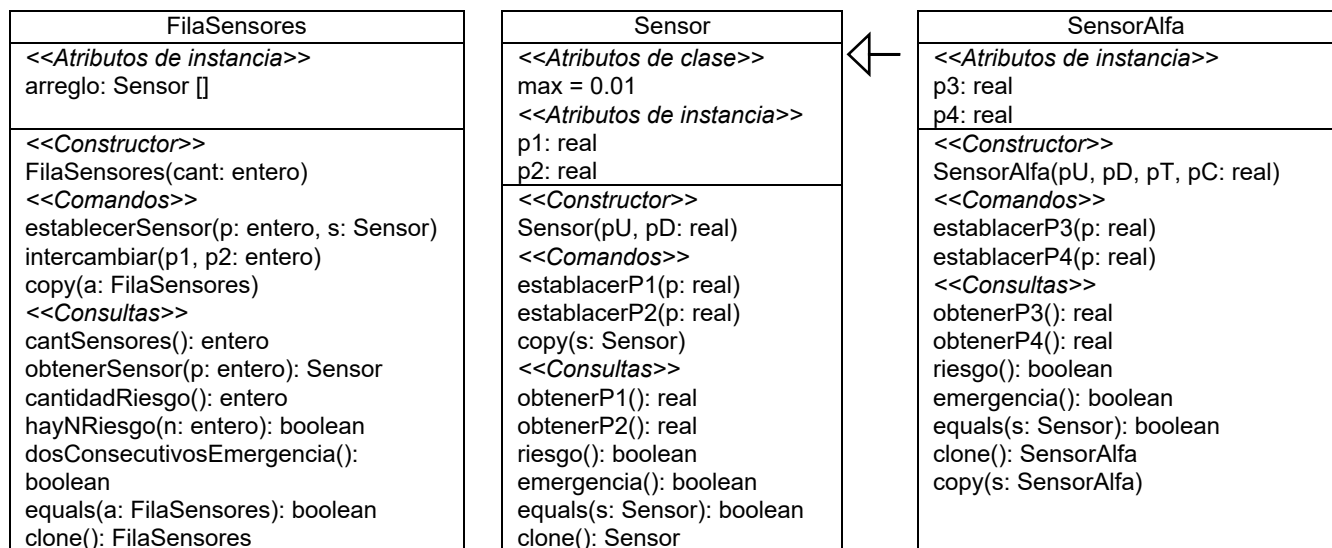
EJERCICIO 1. Implemente y verifique los servicios de la clase *Staff* modelada en el siguiente diagrama:





- *agregarRubrica(r: Rubrica): boolean*. Si *r* está ligada y la colección no está llena, asigna *r* a la posición *cant*, incrementa *cant* en 1 y retorna true, sino retorna false.
- *eliminar(p: entero)*. Asigna nulo a todas las posiciones de la colección que están ligadas a rubricas con productividad menor a *p* y luego comprime la estructura para que continúe siendo una colección.
- *obtenerRubrica(p: entero): Rubrica*. Requiere *p* válida, retorna la rúbrica ligada a la posición *p* de la colección.
- *contarMayorP(p: entero): entero*. Cuenta la cantidad de rubricas con productividad mayor a *p*.
- *totalMayorT(t: entero): Staff*. Genera una nueva colección con el mismo tamaño que la colección que recibe el mensaje y agrega las referencias a todas las rubricas cuyo total es mayor a *t*.

EJERCICIO 2. Implemente y verifique el siguiente diagrama de clases:

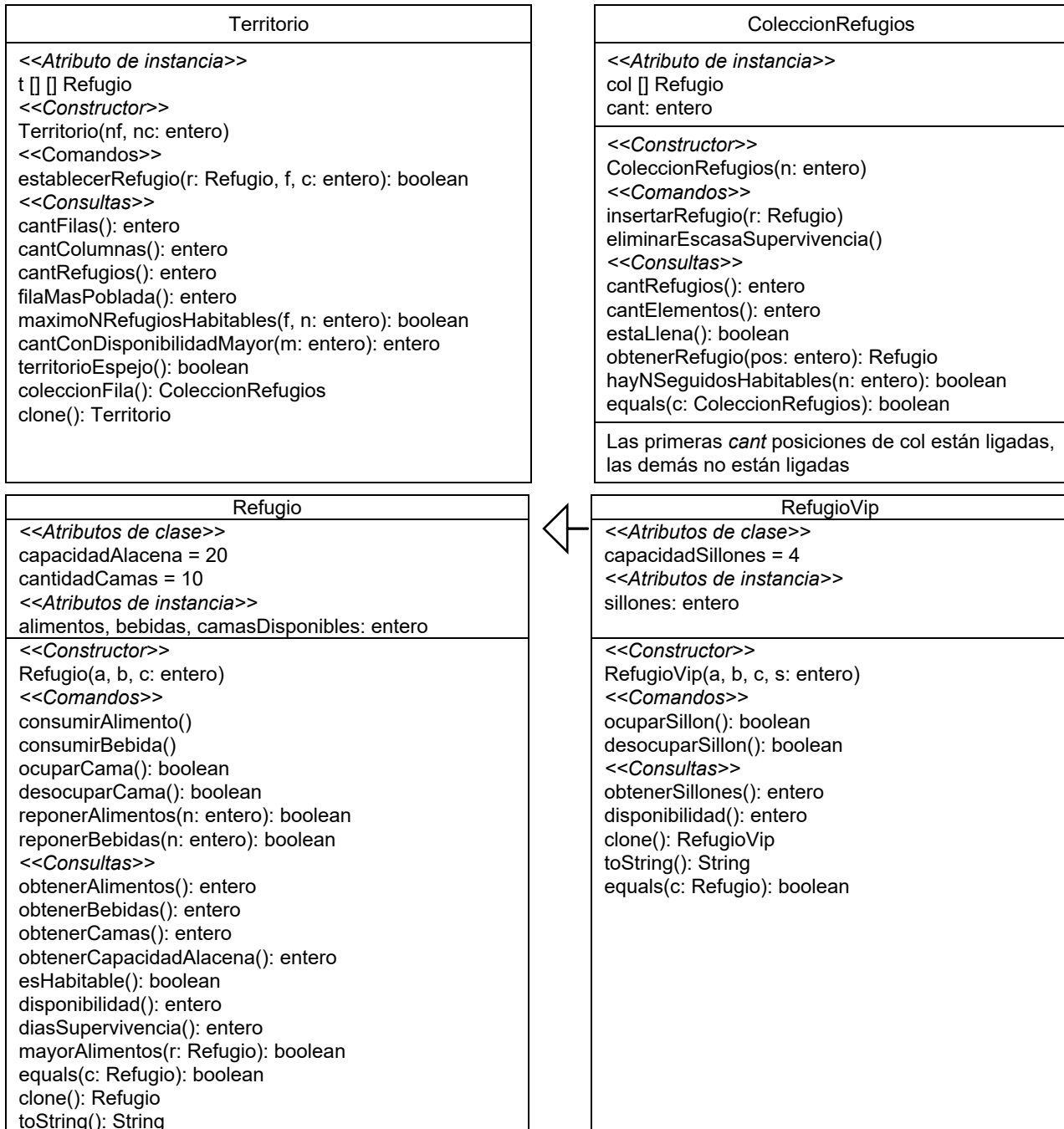


Las clases *FilaSensores* y *Sensor* han sido especificadas antes. En *SensorAlfa*:

- *riesgo(): boolean*. Retorna true si *p2* es mayor a *p1* o bien *p4* es mayor a *p3*.
- *emergencia(): boolean*. Se considera en emergencia cuando *p4* es menor a *max*.
- *equals(s: Sensor): boolean*. Retorna verdadero si *s* está ligado a un objeto de clase *SensorAlfa* y el estado interno del objeto que recibe el mensaje es igual al estado interno de *s*.



EJERCICIO 3. Analice qué cambios debe realizar en la implementación de las clases *Territorio* y *ColeccionRefugios* al extender la clase *Refugio* con la clase derivada *RefugioVip*.



En la clase *Territorio* el constructor requiere *nf* y *nc* mayores a 0.

- *establecerRefugio(r: Refugio, f, c: entero): boolean*. Asigna el refugio *r* a la posición *f, c* y retorna true si $0 \leq f < \text{cantFilas}()$ y $0 \leq c < \text{cantColumnas}()$, sino retorna false.
- *cantFilas(): entero*. Retorna la cantidad de filas del arreglo.
- *cantColumnas(): entero*. Retorna la cantidad de columnas.



- *cantRefugios()*: *entero*. Retorna la cantidad de componentes ligadas del arreglo.
- *filaMasPoblada()*: *entero*. Retorna la fila con mayor número de refugios, si hay más de una con ese valor retorna la primera.
- *maximoNRefugiosHabitables(f, n: entero)*: *boolean*. Retorna true si la fila *f* contiene, a lo sumo, *n* refugios habitables.
- *cantConDisponibilidadMayor(m: entero)*: *entero*. Computa cuántas columnas tienen un refugio con disponibilidad mayor a *m*.
- *territorioEspejo()*: *boolean*. Retorna true si la cantidad de columnas es impar y el territorio a la derecha de la columna central es la imagen espejo del territorio a la izquierda de la columna central, comparando el estado interno de los refugios.
- *coleccionFila()*: *ColeccionRefugios*. Crea y retorna una colección con la primera referencia ligada de cada fila del territorio que recibe el mensaje. Si una fila del territorio no tiene ninguna referencia ligada, no se inserta ningún refugio correspondiente a esa fila.
- *clone()*: *Territorio*. Implementa clone en profundidad.

En la clase *ColeccionRefugios*, el constructor requiere el parámetro mayor a 0.

- *insertarRefugio(r: Refugio)*. Si la colección no está llena, asigna el refugio *r* a la primera posición libre en la colección e incrementa la cantidad de posiciones ligadas. Requiere *r* ligado.
- *eliminarEscasaSupervivencia()*. Elimina de la colección todos los refugios que tienen a lo sumo 1 día de supervivencia y arrastra de modo que el arreglo siga modelando a una colección.
- *obtenerRefugio(pos: entero)*: *Refugio*. Si $0 \leq pos < cantRefugios()$ retorna el refugio de la posición *pos* sino retorna nulo.
- *hayNSeguidosHabitables(n: entero)*: *boolean*. Retorna true si la colección mantiene al menos *n* refugios habitables en posiciones consecutivas.
- *equals(c: ColeccionRefugios)*: *boolean*. Se implementa en profundidad, es decir, retorna true si las dos colecciones tienen el mismo tamaño, la misma cantidad de refugios y los refugios que ocupan la misma posición en los dos territorios, son equivalentes.



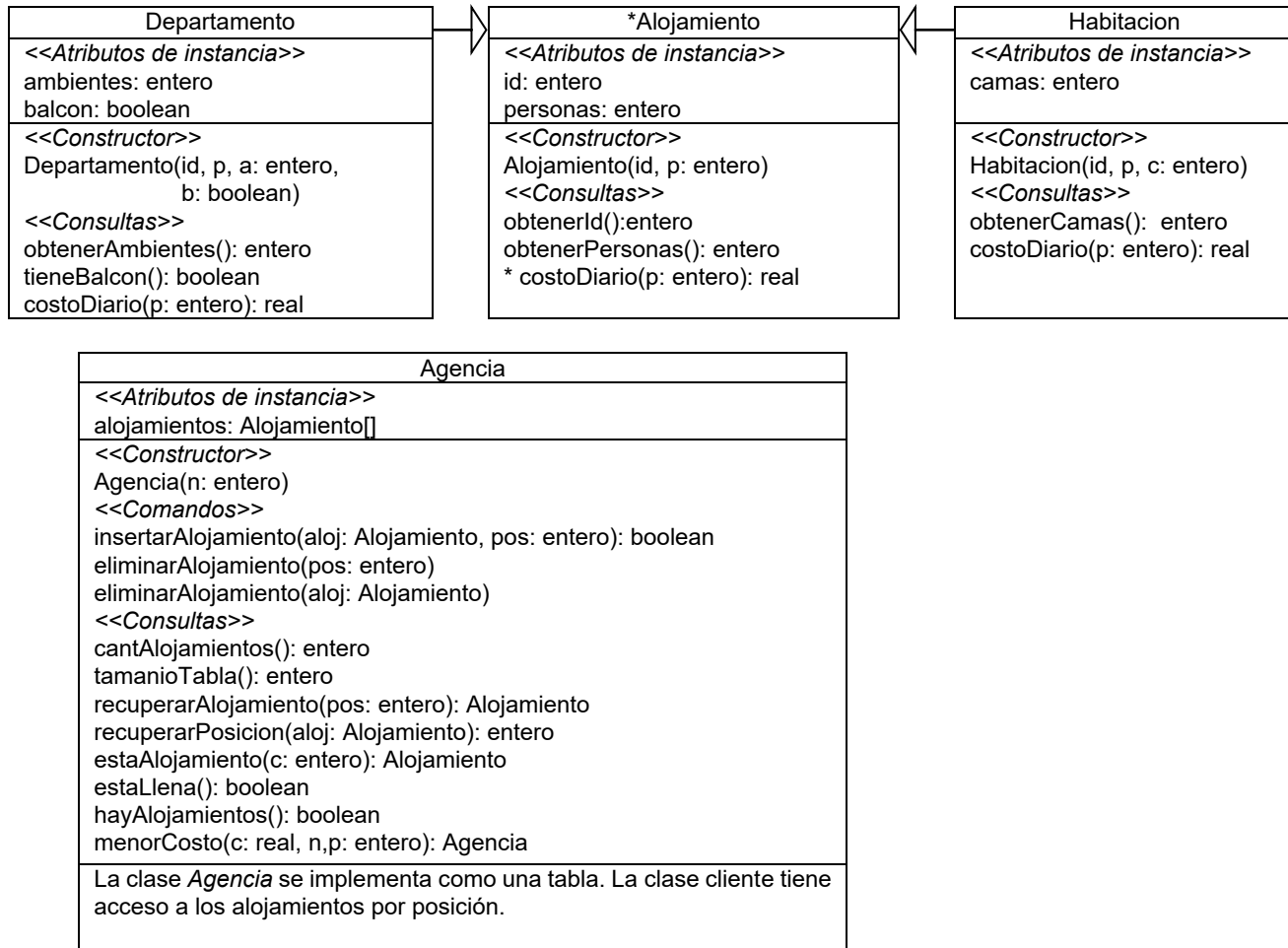
EJERCICIO 4. Dada la clase *Figura* y sus clases derivadas definidas antes, implemente y verifique *Grilla*:

Grilla
<<atributos de instancia>> g: Figura [][]
<<Constructor>> Grilla(cantFil, cantCol: entero)
<<Comandos>> establecerFigura(f ,c: entero, fig: Figura) intercambiar(f1, f2: entero)
<<Consultas>> cantFilas(): entero cantColumnas(): entero obtenerFigura(f, c: entero): Figura cantidadFiguras(): entero cantidadFiguras(p, r: float): entero todosMayores(x: real): boolean contarCoincidenciasArea(gr: Grilla): entero

- *establecerFigura(f, c: entero, fig: Figura)*. Requiere $0 \leq f < \text{cantFilas}()$ y $0 \leq c < \text{cantColumnas}()$.
- *intercambiar(f1, f2: entero)*. Requiere $0 \leq f1, f2 < \text{cantFilas}()$. Intercambia las referencias de las filas *f1* y *f2*.
- *obtenerFigura(f, c: entero): Figura*. Requiere $0 \leq f < \text{cantFilas}()$ y $0 \leq c < \text{cantColumnas}()$.
- *cantidadFiguras(): entero*. Retorna la cantidad de posiciones ligadas de la grilla.
- *cantidadFiguras(p, r: real): entero*. Retorna la cantidad de figuras con perímetro en el rango $[p-r, p+r]$.
- *todosMayores(x: real): boolean*. Retorna true si todas las figuras tienen el área mayor a *x*.
- *contarCoincidenciasArea(gr: Grilla): entero*. Retorna la cantidad de figuras que, ocupando la misma posición en las dos grillas, tienen el mismo área, aunque se trate de figuras diferentes.



EJERCICIO 5. Implemente y verifique el siguiente diagrama de clases:



En la clase *Agencia*:

- *insertarAlojamiento(aloj: Alojamiento, pos: entero)*. Si la tabla no contiene un alojamiento con el mismo id que *aloj*, asigna *aloj* a la posición *pos* y retorna true, sino false. Requiere *pos* valida y *aloj* ligada.
- *eliminarAlojamiento(pos: entero)*. Asigna nulo a la posición *pos*. Requiere *pos* valida.
- *eliminarAlojamiento(aloj: Alojamiento)* Busca la posición de un alojamiento con el mismo id que *aloj* y si existe asigna nulo a esa posición. Requiere *aloj* ligada.
- *cantAlojamientos()*: entero. Retorna la cantidad de posiciones ligadas.
- *tamanoTabla()*: entero. Retorna el tamaño de la estructura.
- *recuperarAlojamiento(pos: entero): Alojamiento*. Retorna la referencia asignada a la posición *pos* de la tabla. Requiere *pos* valida.
- *recuperarPosicion(aloj: Alojamiento): entero*. Busca la posición de un alojamiento con el mismo id que *aloj* y si existe retorna su posición, sino retorna -1. Requiere *aloj* ligada.
- *estaAlojamiento(c: entero): Alojamiento*. Busca un alojamiento con id igual a *c*, si existe retorna la referencia y si no retorna nulo.



- *estaLlena()*: *boolean*. Retorna true si todas las posiciones de la tabla están ligadas.
- *hayAlojamientos()*: *boolean*. Retorna true si al menos una posición de la tabla está ligada.
- *menorCosto(c: real, n, p: entero): Agencia*. Crea una nueva agencia con el mismo tamaño que la que recibe el mensaje, si la agencia que recibe el mensaje contiene al menos una secuencia de exactamente *n* posiciones consecutivas con alojamientos con costo mayor a *c*, inserta esta secuencia de alojamientos en las primeras *n* posiciones de la nueva agencia. El parámetro *p*, contiene el valor utilizado en el cómputo de *costoDiario*. En caso contrario retorna nulo. Si la agencia que recibe el mensaje contiene varias subsecuencias que verifican la propiedad, inserta la primera.